Exercise – 1

Name: Sparsh Karna

Registration Number: 23BDS1172

1. Ceaser Cipher:

a. Server:

```
import socket



def encrypt(s, n):
new_s = ""
for c in s:
if 'a' <= c <= 'z':
new_s += chr((ord(c) - ord('a') + n) % 26 + ord('a'))
elif 'A' <= c <= 'Z':
new_s += chr((ord(c) - ord('A') + n) % 26 + ord('A'))
else:
new_s += c
return new_s



def main():
message = input("Enter message: ")
n = int(input("Enter key: "))

encrypted = encrypt(message, n)

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind(('localhost', 5000))
server_socket.listen(1)
print("Server started. Waiting for client...")

conn, addr = server_socket.accept()
print("Client connected.")

conn.sendall((encrypted + "\n").encode())
```

```
print("Encrypted message sent:", encrypted)

conn.close()
server_socket.close()


if __name__ == "__main__":
main()
```

b. Client:

```
import socket


def decrypt(s, n):
new_s = ""
for c in s:
if 'a' <= c <= 'z':
new_s += chr((ord(c) - ord('a') - n + 26) % 26 + ord('a'))
elif 'A' <= c <= 'Z':
new_s += chr((ord(c) - ord('A') - n + 26) % 26 + ord('A'))
else:
new_s += c
return new_s


def main():
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 5000))

encrypted = client_socket.recv(1024).decode().strip()

print("Encrypted message received:", encrypted)

n = int(input("Enter key: "))

decrypted = decrypt(encrypted, n)
print("Decrypted (original) message:", decrypted)

client_socket.close()


if __name__ == "__main__":
main()
```

c. Output:

```
❯ python3 server.py
Enter message: hello
Enter key: 2
Server started. Waiting for client...
Client connected.
Encrypted message sent: jgnnq

~/dev/sem6/crypto-lab/lab1/ceaser main !9 ?1
❯
```

```
ceaser — sparshkarna@Sparshs-MacBook-Air — ..b/lab1/
❯ python3 client.py
Encrypted message received: jgnnq
Enter key: 2
Decrypted (original) message: hello

~/dev/sem6/crypto-lab/lab1/ceaser main !9 ?1
❯
```

2. Vignere:

a. Server

```
import socket



def encode(plain, key):
"""Encode using Vigenere cipher"""
cipher = ""
for i in range(len(plain)):
pi = plain[i]
ki = key[i % len(key)]
cipher += chr(((ord(pi) - ord('A') + ord(ki) - ord('A')) % 26) + ord('A'))
return cipher
```

```
def main():
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind(('localhost', 9999))
server_socket.listen(1)
print("Server started. Waiting for client...")

conn, addr = server_socket.accept()
print("Client connected.")

plain_text = input("Enter plaintext (uppercase letters only): ").upper()
key = input("Enter key (uppercase letters only): ").upper()

cipher = encode(plain_text, key)

conn.sendall((cipher + "\n").encode())

print("Ciphertext sent to client:", cipher)

conn.close()
server_socket.close()


if __name__ == "__main__":
main()
```

b. Client

```
import socket


def decode(cipher, key):
"""Decode using Vigenere cipher"""
plain = ""
for i in range(len(cipher)):
ci = cipher[i]
ki = key[i % len(key)]
plain += chr(((ord(ci) - ord('A') - (ord(ki) - ord('A')) + 26) % 26) + ord('A'))
return plain


def main():
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
client_socket.connect(('localhost', 9999))
print("Connected to server.")

cipher = client_socket.recv(1024).decode().strip()

print("Received ciphertext:", cipher)

key = input("Enter the key to decipher: ").upper()

plain_text = decode(cipher, key)
print("Deciphered plaintext:", plain_text)

client_socket.close()



if __name__ == "__main__":
main()
```

c. Output

```
❯ python3 server.py
Server started. Waiting for client...
Client connected.
Enter plaintext (uppercase letters only): HELLO
Enter key (uppercase letters only): HI
Ciphertext sent to client: OMSTV

~/dev/sem6/crypto-lab/lab1/vigenere main !9 ?1
❯
```

```
vigenere — sparshkarna@Sparshs-MacBook-Air — ..lab1/

❯ python3 client.py
Connected to server.
Received ciphertext: OMSTV
Enter the key to decipher: HI
Deciphered plaintext: HELLO

~/dev/sem6/crypto-lab/lab1/vigenere main !9 ?1
❯
```

3. Vernam:

a. Server

```
import socket



def encode(plain, key):
"""Encode using Vernam cipher (XOR)"""
cipher = ""
for i in range(len(plain)):
pc = chr(((ord(plain[i]) - ord('A')) ^ (ord(key[i]) - ord('A'))) % 26 + ord('A'))
cipher += pc
return cipher



def main():
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind(('localhost', 1234))
server_socket.listen(1)
```

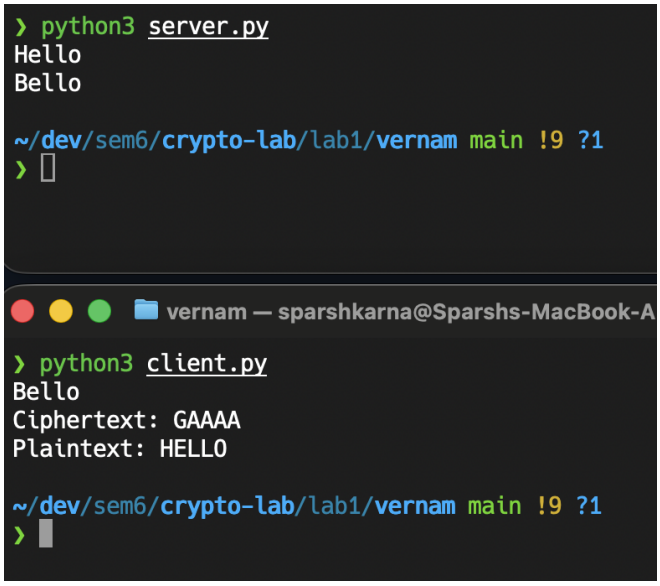
```
conn, addr = server_socket.accept()

plain = input().strip().upper()
key = input().strip().upper()

ciphertext = encode(plain, key)

conn.sendall((ciphertext + "\n").encode())

conn.close()
server_socket.close()



if __name__ == "__main__":
main()
```

b. Client

```
import socket



def decode(cipher, key):
"""Decode using Vernam cipher (XOR)"""
plain = ""
for i in range(len(cipher)):
pc = chr(((ord(cipher[i]) - ord('A')) ^ (ord(key[i]) - ord('A'))) % 26 + ord('A'))
plain += pc
return plain



def main():
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 1234))

ciphertext = client_socket.recv(1024).decode().strip()

key = input().strip().upper()

plaintext = decode(ciphertext, key)

print("Ciphertext:", ciphertext)
print("Plaintext:", plaintext)

client_socket.close()
```

```
if __name__ == "__main__":
main()
```

c. Output



4. Playfair

a. Server

```
import numpy as np



def make_table(key):
"""Create 5x5 Playfair key table"""
key_table = np.empty((5, 5), dtype=str)
used = [False] * 26
row, col = 0, 0

for c in key:
if c < 'A' or c > 'Z':
continue
if c == 'J':
c = 'I'
ci = ord(c) - ord('A')
if not used[ci]:
key_table[row][col] = c
```

```
used[ci] = True
if c == 'I':
used[ord('J') - ord('A')] = True
col += 1
if col == 5:
col = 0
row += 1

for c in range(ord('A'), ord('Z') + 1):
ch = chr(c)
if ch == 'J':
continue
ci = c - ord('A')
if not used[ci]:
key_table[row][col] = ch
used[ci] = True
col += 1
if col == 5:
col = 0
row += 1

return key_table



def find_position(ch, key_table):
"""Find position of character in key table"""
if ch == 'J':
ch = 'I'
for i in range(5):
for j in range(5):
if key_table[i][j] == ch:
return (i, j)
return None



def encode(plain, key_table):
"""Encode plaintext using Playfair cipher"""
plain = plain.upper()

# Remove non-alphabetic characters and replace J with I
prepared = ""
for c in plain:
if 'A' <= c <= 'Z':
if c == 'J':
c = 'I'
prepared += c

# Create digraphs
digraphs = ""
i = 0
while i < len(prepared):
```

```
a = prepared[i]
digraphs += a
if i == len(prepared) - 1:
digraphs += 'X'
break
b = prepared[i + 1]
if a == b:
digraphs += 'X'
else:
digraphs += b
i += 1
i += 1

# Encode digraphs
cipher = ""
for i in range(0, len(digraphs), 2):
curr = digraphs[i]
nex = digraphs[i + 1]
ci = find_position(curr, key_table)
ni = find_position(nex, key_table)

if ci[0] == ni[0]: # Same row
cipher += key_table[ci[0]][(ci[1] + 1) % 5]
cipher += key_table[ni[0]][(ni[1] + 1) % 5]
elif ci[1] == ni[1]: # Same column
cipher += key_table[(ci[0] + 1) % 5][ci[1]]
cipher += key_table[(ni[0] + 1) % 5][ni[1]]
else: # Rectangle
cipher += key_table[ci[0]][ni[1]]
cipher += key_table[ni[0]][ci[1]]

return cipher



def main():
key = input().strip().upper()
plain = input().strip()

key_table = make_table(key)
cipher = encode(plain, key_table)
print(cipher)



if __name__ == "__main__":
main()
```

b. Output

```
❯ python3 server.py
Catastrophic
Hello
[['C' 'A' 'T' 'S' 'R']
 ['O' 'P' 'H' 'I' 'B']
 ['D' 'E' 'F' 'G' 'K']
 ['L' 'M' 'N' 'Q' 'U']
 ['V' 'W' 'X' 'Y' 'Z']]
PFNVVD

~/dev/sem6/crypto-lab/lab1/playfair main !10 ?1
❯
```

~

5. Hill:

a. Server

```
import numpy as np
import math
import socket



def mul(p, k):
"""Matrix multiplication with mod 26"""
result = np.dot(k, p) % 26
return result



def make_matrix(text, rows, cols):
"""Create matrix from text string"""
matrix = np.zeros((rows, cols), dtype=int)
idx = 0
for i in range(rows):
for j in range(cols):
if idx >= len(text):
matrix[i][j] = (ord('X') - ord('A')) % 26
else:
matrix[i][j] = (ord(text[idx]) - ord('A')) % 26
idx += 1
return matrix



def encode(result):
```

```
"""Convert matrix back to cipher string"""
cipher = ""
for row in result:
for val in row:
cipher += chr(int(val) + ord('A'))
return cipher



def main():
plain_text = input("Enter plaintext: ").strip()
key = input("Enter key: ").strip()

n = len(plain_text)
if n * n != len(key):
n = math.ceil(math.sqrt(len(key)))

# Create key matrix (n x n)
key_matrix = make_matrix(key, n, n)
print("Key matrix:")
print(key_matrix)

# Create plain text matrix (n x cols)
cols = math.ceil(len(plain_text) / n)
plain_matrix = make_matrix(plain_text, n, cols)

# Multiply: key_matrix (n x n) * plain_matrix (n x cols)
result = mul(plain_matrix, key_matrix)

cipher = encode(result)

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind(('localhost', 5000))
server_socket.listen(1)
print("Server started. Waiting for client...")

conn, addr = server_socket.accept()
print("Client connected.")

conn.sendall((cipher + "\n").encode())

print("Encrypted message sent:", cipher)

conn.close()
server_socket.close()



if __name__ == "__main__":
main()
```

b. Client

```
import numpy as np
import math
import socket



def mod_inverse(a, m):
"""Find modular inverse of a under modulo m using extended Euclidean algorithm"""
def extended_gcd(a, b):
if a == 0:
return b, 0, 1
gcd, x1, y1 = extended_gcd(b % a, a)
x = y1 - (b // a) * x1
y = x1
return gcd, x, y

gcd, x, _ = extended_gcd(a % m, m)
if gcd != 1:
return None # Modular inverse doesn't exist
return (x % m + m) % m



def matrix_mod_inverse(matrix, mod):
"""Find modular inverse of a matrix"""
det = int(round(np.linalg.det(matrix))) % mod
det_inv = mod_inverse(det, mod)
if det_inv is None:
return None

# Calculate adjugate matrix
n = matrix.shape[0]
adjugate = np.zeros((n, n), dtype=int)

for i in range(n):
for j in range(n):
minor = np.delete(np.delete(matrix, i, axis=0), j, axis=1)
cofactor = ((-1) ** (i + j)) * int(round(np.linalg.det(minor)))
adjugate[j][i] = cofactor

inverse = (det_inv * adjugate) % mod
return inverse



def make_matrix(text, rows, cols):
"""Create matrix from text string"""
```

```
matrix = np.zeros((rows, cols), dtype=int)
idx = 0
for i in range(rows):
for j in range(cols):
if idx >= len(text):
matrix[i][j] = (ord('X') - ord('A')) % 26
else:
matrix[i][j] = (ord(text[idx]) - ord('A')) % 26
idx += 1
return matrix



def decode(result):
"""Convert matrix back to plain text string"""
plain = ""
for row in result:
for val in row:
plain += chr(int(val) + ord('A'))
return plain



def main():
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 5000))

cipher = client_socket.recv(1024).decode().strip()

print("Encrypted message received:", cipher)

key = input("Enter key: ").strip()

n = len(cipher)
if n * n != len(key):
n = math.ceil(math.sqrt(len(key)))

# Create key matrix (n x n)
key_matrix = make_matrix(key, n, n)
print("Key matrix:")
print(key_matrix)

# Find inverse of key matrix
key_inverse = matrix_mod_inverse(key_matrix, 26)

if key_inverse is None:
print("Key matrix is not invertible mod 26")
client_socket.close()
return

# Create cipher matrix (n x cols)
cols = math.ceil(len(cipher) / n)
```

```
cipher_matrix = make_matrix(cipher, n, cols)

# Multiply: key_inverse (n x n) * cipher_matrix (n x cols)
result = np.dot(key_inverse, cipher_matrix) % 26

plain = decode(result)
print("Decrypted (original) message:", plain)

client_socket.close()



if __name__ == "__main__":
main()
```

c. Output

```
❯ python3 server.py
Enter plaintext: Crypto
Enter key: Hill
Key matrix:
[[ 7  8]
 [11 11]]
Server started. Waiting for client...
Client connected.
Encrypted message sent: ELUFGC

~/dev/sem6/crypto-lab/lab1/hill main ↑1 !12 ?1
❯
```

```
hill — sparshkarna@Sparshs-MacBook-Air — ..lab/l

❯ python3 client.py
Encrypted message received: ELUFGC
Enter key: Hill
Key matrix:
[[ 7  8]
 [11 11]]
Decrypted (original) message: CRYPTO

~/dev/sem6/crypto-lab/lab1/hill main ↑1 !12 ?1
❯
```

6. Row-Column:

a. Server

```
import socket
import math
import numpy as np



def swap_col(table, i, j):
"""Swap two columns in the table"""
table[:, [i, j]] = table[:, [j, i]]



def sort_table(table, key):
"""Sort columns based on key alphabetical order"""
key_arr = list(key)
n = len(key)
```

```
for i in range(n):
for j in range(n - i - 1):
if key_arr[j] > key_arr[j + 1]:
swap_col(table, j, j + 1)
key_arr[j], key_arr[j + 1] = key_arr[j + 1], key_arr[j]



def encode(table, key):
"""Encode by reading columns after sorting"""
sort_table(table, key)
cipher = ""
rows, cols = table.shape
for c in range(cols):
for r in range(rows):
cipher += table[r][c]
return cipher



def make_table(plain, key):
"""Create the table from plaintext"""
cols = len(key)
rows = math.ceil(len(plain) / cols)
table = np.empty((rows, cols), dtype=str)

idx = 0
for r in range(rows):
for c in range(cols):
if idx < len(plain):
table[r][c] = plain[idx]
idx += 1
else:
table[r][c] = '_'

return table



def main():
plain_text = input("Enter message: ")
key = input("Enter key: ")

table = make_table(plain_text, key)
cipher = encode(table, key)

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind(('localhost', 5000))
server_socket.listen(1)
print("Server started. Waiting for client...")

conn, addr = server_socket.accept()
```

```
print("Client connected.")

conn.sendall((cipher + "\n").encode())

print("Encrypted message sent:", cipher)

conn.close()
server_socket.close()



if __name__ == "__main__":
main()
```

b. Client

```
import socket
import math
import numpy as np



def decrypt(cipher, key):
"""Decrypt by reversing the column sort"""
cols = len(key)
rows = math.ceil(len(cipher) / cols)
# Create table to hold sorted columns
table = np.empty((rows, cols), dtype=str)
# Get the sorted key indices
key_with_indices = [(char, idx) for idx, char in enumerate(key)]
sorted_key = sorted(key_with_indices)
# Fill the table column by column from ciphertext (in sorted key order)
idx = 0
for _, original_col in sorted_key:
for r in range(rows):
if idx < len(cipher):
table[r][original_col] = cipher[idx]
idx += 1
else:
table[r][original_col] = '_'
# Read row by row to get plaintext
plain = ""
for r in range(rows):
for c in range(cols):
plain += table[r][c]
return plain.replace("_", "")



def main():
```

```python
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 5000))

cipher = client_socket.recv(1024).decode().strip()

print("Ciphertext received:", cipher)

key = input("Enter key: ")

plain = decrypt(cipher, key)
print("Decrypted (original) message:", plain)

client_socket.close()



if __name__ == "__main__":
main()
```

c. Output

```
❯ python3 server.py
Enter message: Crypto
Enter key: Hello
Server started. Waiting for client...
Client connected.
Encrypted message sent: Cor_y_p_t_

~/dev/sem6/crypto-lab/lab1/row-column main ↑3 !11 ?1
❯
```

```
row-column — sparshkarna@Sparshs-MacBook-Air — ..b1/row

❯ python3 client.py
Ciphertext received: Cor_y_p_t_
Enter key: Hello
Decrypted (original) message: Crypto

~/dev/sem6/crypto-lab/lab1/row-column main ↑3 !11 ?1
❯
```

7. Railfence

a. Server

```
import socket
import numpy as np



def encrypt(plain, key):
"""Encrypt using rail fence cipher"""
rail = np.zeros((key, len(plain)), dtype=str)
rail[:] = ''

dir_down = True
row = 0

for i, char in enumerate(plain):
rail[row][i] = char
if dir_down:
if row == key - 1:
dir_down = False
row -= 1
else:
row += 1
else:
if row == 0:
dir_down = True
row += 1
else:
row -= 1

cipher = ""
for i in range(key):
for j in range(len(plain)):
if rail[i][j] != '':
cipher += rail[i][j]

return cipher



def main():
plain_text = input("Enter message: ")
key = int(input("Enter key: "))

cipher = encrypt(plain_text, key)

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind(('localhost', 5000))
server_socket.listen(1)
print("Server started. Waiting for client...")
```

```
conn, addr = server_socket.accept()
print("Client connected.")

conn.sendall((cipher + "\n").encode())

print("Encrypted message sent:", cipher)

conn.close()
server_socket.close()



if __name__ == "__main__":
main()
```

b. Client

```
import socket
import numpy as np



def decrypt(cipher, key):
"""Decrypt using rail fence cipher"""
rail = np.zeros((key, len(cipher)), dtype=str)
rail[:] = ''

# Mark the positions
dir_down = True
row = 0
for i in range(len(cipher)):
rail[row][i] = '*'
if dir_down:
if row == key - 1:
dir_down = False
row -= 1
else:
row += 1
else:
if row == 0:
dir_down = True
row += 1
else:
row -= 1

# Fill the cipher characters
index = 0
for i in range(key):
```

```
for j in range(len(cipher)):
if rail[i][j] == '*':
rail[i][j] = cipher[index]
index += 1

# Read in zig-zag pattern
plain = ""
dir_down = True
row = 0
for i in range(len(cipher)):
plain += rail[row][i]
if dir_down:
if row == key - 1:
dir_down = False
row -= 1
else:
row += 1
else:
if row == 0:
dir_down = True
row += 1
else:
row -= 1

return plain



def main():
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 5000))

encrypted = client_socket.recv(1024).decode().strip()

print("Encrypted message received:", encrypted)

n = int(input("Enter key: "))

decrypted = decrypt(encrypted, n)
print("Decrypted (original) message:", decrypted)

client_socket.close()



if __name__ == "__main__":
main()
```

c. Output

```
❯ python3 server.py
Enter message: Crypto
Enter key: 3
Server started. Waiting for client...
Client connected.
Encrypted message sent: Ctrpoy

~/dev/sem6/crypto-lab/lab1/railfence main ↑3 !11 ?1
❯
```

railfence — sparshkarna@Sparshs-MacBook-Air — ..ab1/railfence

```
❯ python3 client.py
Encrypted message received: Ctrpoy
Enter key: 3
Decrypted (original) message: Crypto

~/dev/sem6/crypto-lab/lab1/railfence main ↑3 !11 ?1
❯
```